# ALGORITMOS DE CLASIFICACIÓN

Andrés Camilo Patiño Serna

Jhonattan Pineda Cardona

Yansit Stiven Rodriguez Diaz

CAI

Introducción a Machine Learning

Docente: Mónica María Fonseca Vigoya

OCTUBRE 2026

## Planteamiento del caso

El presente trabajo tiene como objetivo comparar cinco algoritmos de clasificación de Machine Learning para predecir si un tumor de mama es benigno o maligno, utilziando el conjunto de datos Breast Cancer Wisconsin Diagnostic. Estos algoritmos se evaluaron mediante las métricas Accuracy, Precision, Recall y F1-score, para determinar cuál obtuvo el mejor desempeño.

## Preparación de los datos

Se utilizó el conjunto de datos Breast Cancer Wisconsin Diagnostic, disponible en la biblioteca Scikit-learn, compuesto por 569 muestras y 30 variables predictoras. La variable objetivo se codificó como 1 para los tumores malignos y 0 para los benignos. Los datos se dividieron en un 80 % para entrenamiento y un 20 % para prueba, utilizando random_state=42 y stratify=y para mantener una distribución similar de las clases. Los algoritmos SVM y K-NN utilizaron estandarización de las variables, mientras que los demás se entrenaron sin escalamiento.

## Ejercicio 1 - Preparación del caso y los datos

Crea un notebook nuevo en Google Colab. Utiliza el siguiente código únicamente como punto de partida y completa las celdas de análisis que se solicitan.

**Variable objetivo:** La variable objetivo corresponde a la clasificación del tumor, codificada como 1 para los tumores malignos y 0 para los benignos. Las 30 características restantes se utilizaron como variables predictoras para entrenar y evaluar los algoritmos de clasificación.

```python
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Cargar el dataset en formato DataFrame
data = load_breast_cancer(as_frame=True)

# Obtener las variables predictoras
X = data.data.copy()

# Convertir la clase positiva en 'maligno'
y = (data.target == 0).astype(int)

# Mostrar las dimensiones y la distribución de las clases
print("Dimensiones de X:", X.shape)
print("\nDistribución de las clases:")
print(y.value_counts())
```

**Resultados:**

```
Dimensiones de X: (569, 30)

Distribución de las clases:
target
0    357
1    212
Name: count, dtype: int64
```

**¿Cuántas observaciones y variables contiene el conjunto de datos?**

El conjunto de datos contiene 569 observaciones y 30 variables predictoras. Cada observación representa una muestra analizada y las variables son mediciones de imágenes digitalizadas. Además de esto, el conjunto cuanta con una variable objetivo que representa el diagnóstico maligno o benigno.

**¿La variable objetivo está balanceada? Explica brevemente lo que observas.**

La variable objetivo no está del todo balanceada, contiene 357 casos benignos (0) y 212 casos malignos (1), lo que indica que aproximadamente el 62,74 % de las muestras son benignas y el 37,26 % son malignas. Existe una diferencia entre las clases, pero ninguna es minoritaria.

**¿Por qué es importante definir con claridad qué representa la clase positiva?**

Es importante definir la clase positiva porque permite interpretar las métricas de evaluación del modelo, en especial la precisión (Precision), la sensibilidad (Recall) y la medida F1-score. En este caso, la clase positiva corresponde a los casos malignos (1), se puede evaluar entonces cuántos de estos casos el modelo identifica y cuántos clasifica erróneamente como benignos.

División entrenamiento / prueba

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(X_train.shape, X_test.shape)
```

**Resultados:**

```
(455, 30) (114, 30)
```

**¿Qué función cumple stratify=y en esta división?**

El parámetro stratify=y permite mantener aproximadamente la misma proporción de las clases de la variable objetivo en los conjuntos de entrenamiento y prueba; en este caso ayuda a conservar una distribución similar de casos benignos y malignos en ambos conjuntos.

### Función común de evaluación

Puedes construir una función reutilizable para evitar repetir código. Complétala y verifica que muestre las cinco métricas solicitadas.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

def evaluar_modelo(nombre, modelo, X_eval, y_eval):
    pred = modelo.predict(X_eval)

    resultados = {
        'Modelo': nombre,
        'Accuracy': accuracy_score(y_eval, pred),
        'Precision': precision_score(y_eval, pred, zero_division=0),
        'Recall': recall_score(y_eval, pred, zero_division=0),
        'F1': f1_score(y_eval, pred, zero_division=0)
    }

    print("Modelo:", nombre)
    print("\nMatriz de confusión:")
    print(confusion_matrix(y_eval, pred))

    print("\nMétricas de evaluación:")
    for metrica, valor in resultados.items():
        if metrica != 'Modelo':
            print(f"{metrica}: {valor:.4f}")

    return resultados
```

## Ejercicios 2 a 6 - Implementación de los clasificadores

### Ejercicio 2 · Decision Tree

Entrena un Árbol de Decisión. Utiliza max_depth=4 para conservar un modelo relativamente interpretable.

**Resultado:**

```
Modelo: Decision Tree


Matriz de confusión:
[[70  2]
 [ 8 34]]


Métricas de evaluación:
Accuracy: 0.9123
Precision: 0.9444
Recall: 0.8095

F1: 0.8718
```

Esto significa:

Precision (Confiabilidad de las predicciones positivas del modelo): 94,44 %

Recall (Capacidad de un modelo de clasificación): 80,95 %

F1-Score (Rendimiento del modelo de clasificación): 87,18 %

Casos de prueba (Conjunto de datos): 114

Se concluye que:

Clasificó correctamente 70 casos benignos

Clasificó erróneamente 2 casos benignos como malignos

Identificó correctamente 34 casos malignos

Clasificó erróneamente 8 casos malignos como benignos

**Requisitos en el primer algoritmo, Decision Tree:**

**1. Identificar la variable objetivo y las variables de entrada:**

X contiene las 30 variables predictoras y representa el diagnóstico: 1 = maligno y 0 = benigno. Se definió en el primer ejercicio.

**2. Preparar los datos cuando sea necesario:**

Se Cargaron los datos, definiendo la variable objetivo y se realizó la división. El Árbol de Decisión no necesita escalamiento para este ejercicio.

**3. Utilizar el mismo conjunto de entrenamiento y prueba:**

Se ha utilizado:  X_train, X_test, y_train y y_test, creados previamente con una división 80/20.

**4. Entrenar el modelo:**

Se ha entrenado con:  modelo_dt.fit(X_train, y_train).

**5. Realizar predicciones:**

La función evaluar_modelo() ejecutó modelo.predict(X_test) automáticamente.

**6. Evaluar con las cinco métricas solicitadas**

Se obtuvo Accuracy = 0.9123, Precision = 0.9444, Recall = 0.8095, F1 = 0.8718 y la matriz de confusión.

**¿Qué ventaja tiene limitar la profundidad del árbol?**

Limitar la profundidad del árbol ayuda a reducir el sobreajuste (overfitting), es decir, que el modelo aprenda demasiado y facilita la interpretación de las decisiones de clasificación. En este ejercicio, se establece max_depth=4, lo cual permite conservar un modelo relativamente sencillo.

**¿Qué errores observas en la matriz de confusión?**

En la matriz de confusión se observan 2 falsos positivos que corresponden a casos benignos clasificados como malignos y 8 falsos negativos, que son casos malignos clasificados como benignos.

**Ejercicio 3 · Random Forest**

Entrena un Random Forest con 100 árboles y compara sus resultados con el Árbol de Decisión.

```
from sklearn.ensemble import RandomForestClassifier

# Crear el modelo con 100 árboles

modelo_rf = RandomForestClassifier(

    n_estimators=100,

    random_state=42

)

# Entrenar el modelo con los mismos datos de entrenamiento

modelo_rf.fit(X_train, y_train)

# Evaluar el modelo con los mismos datos de prueba

resultados_rf = evaluar_modelo(

    "Random Forest",

    modelo_rf,

    X_test,

    y_test

)
```

**Resultados:**

```
Modelo: Random Forest


Matriz de confusión:
[[72  0]
 [ 3 39]]


Métricas de evaluación:
Accuracy: 0.9737
Precision: 1.0000
Recall: 0.9286

F1: 0.9630
```

Esto significa:

Precision (Confiabilidad de las predicciones positivas del modelo): 100 %

Recall (Capacidad de un modelo de clasificación): 92,86 %

F1-Score (Rendimiento del modelo de clasificación): 96,30 %

Falsos negativos: 3

Se concluye que:

Clasificó correctamente 72 casos benignos

Clasificó erróneamente 0 casos benignos como malignos

Identificó correctamente 39 casos malignos

Clasificó erróneamente 3 casos malignos como benignos

**Requisitos en el segundo algoritmo, Random Forest:**

**1. Identificar la variable objetivo y las variables de entrada:**

Se utilizó `X`, que contiene las 30 variables predictoras, Y, que representa el diagnóstico: 1 = maligno y 0 = benigno. Son las mismas variables que se usaron en Decision Tree.

**2. Preparar los datos cuando sea necesario:**

Los datos se han sido cargados, organizados y divididos en el primer ejercicio. Random Forest no necesita escalamiento para este caso.

**3. Utilizar el mismo conjunto de entrenamiento y prueba:**

Se ha utilizado `X_train`, `y_train`, `X_test` y `y_test`, conservando la división original del 80 % para entrenamiento y 20 % para prueba.

**4. Entrenar el modelo:**

Se entrenó el clasificador mediante `modelo_rf.fit(X_train, y_train)`, utilizando 100 árboles con `n_estimators=100`.

**5. Realizar predicciones:**

La función `evaluar_modelo()` ejecutó automáticamente `modelo_rf.predict(X_test)` para generar las predicciones.

**6. Evaluar con las cinco métricas solicitadas**

Se obtuvo Accuracy = 0.9737, Precision = 1.0000, Recall = 0.9286, F1-score = 0.9630 y la matriz de confusión.

**¿Qué cambia al pasar de un árbol individual a un conjunto de árboles?**

Al pasar de un árbol de decisión individual a Random Forest, se utilizan múltiples árboles, lo cual permite combinar diferentes resultados y reducir e riesgo de sobreajuste.

**¿Mejora alguna métrica?, ¿Cuál y cuánto?**

Sí, han mejorado cuatro métricas Accuracy , de 91,23 % , Precision , de 94,44 % a 100 % , Recall, de 80,95 % a 92,86 % y F1-Score , de 87,18 % a 96,30 %.

**Ejercicio 4 · Support Vector Machine (SVM)**

SVM es sensible a la escala de las variables. Utiliza un Pipeline para estandarizar los datos antes de entrenar el clasificador.

from sklearn.pipeline import make_pipeline

from sklearn.preprocessing import StandardScaler

```
from sklearn.svm import SVC

modelo_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1.0, gamma='scale')
)

# Entrenar el modelo
modelo_svm.fit(X_train, y_train)



# Evaluar el modelo
resultados_svm = evaluar_modelo(
    "Support Vector Machine (SVM)",
    modelo_svm,
    X_test,
    y_test
)
```

**Resultados:**

```
Modelo: Support Vector Machine (SVM)


Matriz de confusión:
[[72  0]
 [ 3 39]]


Métricas de evaluación:
Accuracy: 0.9737
Precision: 1.0000
Recall: 0.9286

F1: 0.9630
```

Esto significa:

Precision (Confiabilidad de las predicciones positivas del modelo): 100 %

Recall (Capacidad de un modelo de clasificación): 92,86 %

F1-Score (Rendimiento del modelo de clasificación): 96,30 %

Accuracy: 97,37 %

Se concluye que:

Clasificó correctamente 72 casos benignos

Clasificó erróneamente 0 casos benignos como malignos

Identificó correctamente 39 casos malignos

Clasificó erróneamente 3 casos malignos como benignos

Se obtuvieron las mismas métricas que el Ransom Forest.

**Requisitos en el tercer algoritmo, SVM:**

**1. Identificar la variable objetivo y las variables de entrada:**

utiliza las variables X y la etiqueta y definidas previamente.

**2. Preparar los datos cuando sea necesario:**

Tiene escalado, `StandardScaler()` se aplica antes de SVM mediante `make_pipeline`.

**3. Utilizar el mismo conjunto de entrenamiento y prueba:**

reutiliza `X_train`, `X_test`, `y_train` y `y_test`.

**4. Entrenar el modelo:**

Se entrena mediante `modelo_svm.fit(X_train, y_train)`.

**5. Realizar predicciones:**

Calcula Accuracy, Precision, Recall, F1-score y la matriz de confusión.

**6. Evaluar con las cinco métricas solicitadas:**

Se obtuvo Precision (Confiabilidad de las predicciones positivas del modelo): 100 %, Recall (Capacidad de un modelo de clasificación): 92,86 %,F1-Score (Rendimiento del modelo de clasificación): 96,30 %, Accuracy: 97,37 %.

**¿Por qué en este modelo usamos StandardScaler?**

Se utiliza StandardScaler porque SVM es sensible a la escala de las variabñes, de este modo las variables tienen una media cercana a 0 y una desviación estándar cercana a 1.

**¿Qué resultado obtiene SVM frente a los modelos basados en árboles?**

SVM obtuvo mejores resultados que Decision Tree e iguales a Random Forest en las métricas, Alcanzó un Accuracy del 97,37 %, una Precision del 100 %, un Recall del 92,86 % y un F1-score del 96,30 %. Frente a Decision Tree, mejoró todas las métricas; frente a Random Forest, obtuvo exactamente los mismos valores. Además, SVM presentó tres falsos negativos y ningún falso positivo en el conjunto de prueba.

**Ejercicio 5 · K-Nearest Neighbors (K-NN)**

Implementa K-NN usando k=5. Mantén el escalamiento dentro del Pipeline para que las distancias no estén dominadas por variables de mayor magnitud.

```
from sklearn.neighbors import KNeighborsClassifier
```

```
modelo_knn = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
# Entrenar el modelo
modelo_knn.fit(X_train, y_train)
# Evaluar el modelo
resultados_knn = evaluar_modelo(
    "K-Nearest Neighbors (K-NN)",
    modelo_knn,
    X_test,
    y_test
)
```

**Resultados:**

```
Modelo: K-Nearest Neighbors (K-NN)


Matriz de confusión:
[[71  1]
 [ 4 38]]


Métricas de evaluación:
Accuracy: 0.9561
Precision: 0.9744
Recall: 0.9048

F1: 0.9383
```

Esto significa:

Precision (Confiabilidad de las predicciones positivas del modelo): 97,44 %

Recall (Capacidad de un modelo de clasificación): 90,48 %

F1-Score (Rendimiento del modelo de clasificación): 93,83 %

Accuracy: 95,61 %

Se concluye que:

Clasificó correctamente 71 casos benignos

Clasificó erróneamente 1 casos benignos como malignos

Identificó correctamente 38 casos malignos

Clasificó erróneamente 4 casos malignos como benignos

**Requisitos en el cuarto algoritmo, neighbor:**

**1. Identificar la variable objetivo y las variables de entrada:**

Utiliza las variables predictoras X y la etiqueta y definidas previamente.

**2. Preparar los datos cuando sea necesario:**

Utiliza `StandardScaler()` dentro del `Pipeline`.

**Número de vecinos:** se configuró con n_neighbors=5.

**3. Utilizar el mismo conjunto de entrenamiento y prueba:**

Reutiliza los mismos conjuntos de entrenamiento y prueba.

**4. Entrenar el modelo:**

Entrena con modelo_knn.fit(X_train, y_train).

**5. Realizar predicciones:**

Se calcularon Accuracy, Precision, Recall, F1-score y la matriz de confusión.

**6. Evaluar con las cinco métricas solicitadas:**

Se obtuvo Precision (Confiabilidad de las predicciones positivas del modelo): 97,44 %, Recall (Capacidad de un modelo de clasificación): 90,48 %,F1-Score (Rendimiento del modelo de clasificación): 93,83 %, Accuracy: 95,61 %.

**¿Qué representa k=5 en este algoritmo?**

El valor k=5 indica que el algoritmo K-Nearest Neighbors utiliza los cinco vecinos más cercanos a una muestra.

**¿Por qué el escalamiento es especialmente importante cuando se calculan distancias?**

El escalamiento es importante porque K-NN calcula distancias entre las muestras, para encontrar sus vecinos más cercanos.

**Ejercicio 6 · Naive Bayes**

Utiliza GaussianNB, apropiado para variables numéricas continuas. En este ejercicio úsalo sin escalamiento para mantener una comparación simple y directa.

```
from sklearn.naive_bayes import GaussianNB

modelo_nb = GaussianNB()

# Entrenar el modelo

modelo_nb.fit(X_train, y_train)

# Evaluar el modelo
```

```
resultados_nb = evaluar_modelo(
    "Gaussian Naive Bayes",
    modelo_nb,
    X_test,
    y_test
)
```

**Resultados:**

```
Modelo: Gaussian Naive Bayes

Matriz de confusión:
[[72  0]
 [ 7 35]]

Métricas de evaluación:
Accuracy: 0.9386
Precision: 1.0000
Recall: 0.8333
```

F1: 0.9091

Esto significa:

Precision (Confiabilidad de las predicciones positivas del modelo): 100 %

Recall (Capacidad de un modelo de clasificación): 83,33 %

F1-Score (Rendimiento del modelo de clasificación): 90,91 %

Accuracy: 93,86 %

Se concluye que:

Clasificó correctamente 72 casos benignos

Clasificó erróneamente 0 casos benignos como malignos

Identificó correctamente 35 casos malignos

Clasificó erróneamente 7 casos malignos como benignos

**Requisitos en el quintio algoritmo, BAYES:**

**1. Identificar la variable objetivo y las variables de entrada:**

Utiliza X como conjunto de variables predictoras, Y como variable objetivo.

**2. Preparar los datos cuando sea necesario:**

No tiene escalamiento.

**3. Utilizar el mismo conjunto de entrenamiento y prueba:**

Reutiliza los mismos conjuntos de entrenamiento y prueba.

**4. Entrenar el modelo:**

Se entrena mediante modelo_nb.fit(X_train, y_train).

**5. Realizar predicciones:**

Se calcularon Accuracy, Precision, Recall, F1-score y la matriz de confusión.

**6. Evaluar con las cinco métricas solicitadas:**

Se obtuvo Precision (Confiabilidad de las predicciones positivas del modelo): 100 %, Recall (Capacidad de un modelo de clasificación): 83,33 %,F1-Score (Rendimiento del modelo de clasificación): 90,91 %, Accuracy: 93,86 %.

**¿Qué supuesto simplificador utiliza Naive Bayes respecto a las variables?**

Naive Bayes supone que las variables son independientes entre sí, lo que significa que, para realizar una clasificación, el algoritmo considera cada variable por separado, simplificando cálculos.

**¿En qué métricas se comporta mejor o peor que los otros clasificadores?**

Naive Bayes obtuvo una precisión del 100 % , como Random Dorest y SVM, lo cual indica que no obtuvo falsos positivos. Sin embargo, el Accuracy (exactitud) fue del 93,86 %, su Recall del 83,33 % y su F1-score del 90,91 %. Estas tres métricas fueron inferiores a las de Random Forest, SVM y K-NN. Además, presentó siete falsos negativos, es decir, siete casos malignos que fueron clasificados como benignos.

**Ejercicio 7 - Comparación y toma de decisión**

Consolida los resultados de los cinco modelos en una sola tabla. No copies únicamente los valores: interprétalos.

| Modelo | Accuracy | Precision | Recall | F1 | Observación |
|---|---|---|---|---|---|
| Decision Tree | 91,23 % | 94,44 % | 80,95 % | 87,18 % | No es 100 % en las características, pero al igual que los demás algritmos, tiene porcentajes altos. |
| Random Forest | 97,37 % | 100 % | 92,86 % | 96,30 % | Se puede decir que es igual con el SVM, reduce el overfitting al tener varios árboles. |
| SVM | 97,37 % | 100 % | 92,86 % | 96,30 % | Predice con la exactitud oportuna. |
| K-NN | 95,61 % | 97,44 % | 90,48 % | 93,83 % | Ronda el 100 %, es un algoritmo confiable. |
| Naive Bayes | 93,86 % | 100 % | 83,33 % | 90,91 % | Predice con la excatitud oportuna, al igual que el Random Forest y el SVM. |

**Nota.** Elaboración propia a partir de los resultados obtenidos al evaluar cinco algoritmos de clasificación en el conjunto de prueba (20 %) del *Breast Cancer Wisconsin Diagnostic Dataset*.

## Interpretación de las matrices de confusión

Las matrices de confusión permitieron identificar los aciertos y errores de cada algoritmo al clasificar los tumores. Random Forest y SVM obtuvieron los mejores resultados, con 39 tumores malignos identificados correctamente y solo 3 casos malignos clasificados erróneamente como benignos. K-NN presentó 4 falsos negativos, Gaussian Naive Bayes 7 y Decision Tree 8. Por tanto, Random Forest y SVM mostraron el mejor desempeño en la identificación de tumores malignos dentro del conjunto de prueba.

## Selección del modelo

El algoritmo seleccionado ha sido Random Forest, ya que, obtuvo un Accuracy de 97,37 % Precision de 100 % , Recall de 92,86 % y F1-score de 96,30 %. Además presentó solo 3 falsos negativos y ningún falso positivo.

## Preguntas de análisis

### ¿Qué modelo obtuvo la mayor Accuracy?

Los modelos Random Forest y Support Vector Machine (SVM) obtuvieron la mayor Accuracy, con un valor de 97,37 %.

**¿Qué modelo obtuvo el mayor Recall para la clase maligna?**

Random Forest y SVM obtuvieron el mayor Recall, con un 92,86 %, esto significa que ambos identificaron correctamente 39 de los 42 casos malignos.

**¿Qué modelo produjo menos falsos negativos?**

Random Forest y SVM produjeron la menor cantidad de falsos negativos, con tres casos cada uno. Esto significa que ambos clasificaron incorrectamente como benignas tres muestras que realmente correspondían a casos malignos.

**¿Coincide el modelo con mayor Accuracy con el modelo que seleccionarías? Explica.**

Sí coincide, el modelo sería Random Forest o SVM, ya que ambos obtuvieron la mayor Accuracy, del 97,37 % y también el mayor Recall, del 92,86 %, junto con F1-score del 96,30 %. Las métricas mencionadas demuestran un buen equilibrio entre el desempeño general y la identificación de casos malignos. Para escoger entre los dos, se tendría en cuenta la facilidad de interpretación y el costo computacional.

**Si el costo de no detectar un caso maligno fuera especialmente alto, ¿qué métrica priorizarías y por qué?**

Priorizaría el Recall de la clase maligna, porque mide la proporción de casos malignos que el modelo identifica correctamente. Un Recall elevado reduce la proporción de falsos negativos, es decir, los casos malignos clasificados erróneamente como benignos. En este ejercicio, Random Forest y SVM obtuvieron el mayor Recall.

**¿Cuál de los modelos es más sencillo de explicar a una persona no técnica?**

El modelo más sencillo de explicar es el de Decision Tree, porque sus decisiones se representan mediante un árbol con preguntas y condiciones para una clasificación. Esto facilita comprender la forma en que predice el modelo.

**¿Qué diferencias observaste entre los algoritmos que requieren escalamiento y los que no?**

SVM y K-NN utilizaron StandardScaler() dentro de un Pipeline, porque ambos son sensibles a la escala de las variables. En SVM, la escala puede afectar la construcción de la frontera de clasificación, mientras que en K-NN influye directamente en el cálculo de distancias. En cambio, Decision Tree, Random Forest y Gaussian Naive Bayes se entrenaron sin escalamiento, siguiendo las instrucciones de la actividad. En este ejercicio, SVM obtuvo resultados iguales a Random Forest, mientras que K-NN alcanzó un desempeño ligeramente inferior.

**Selecciona un modelo final y justifica tu decisión usando al menos dos métricas y una consideración del contexto.**

Seleccionaría Random Forest como modelo final, en sus métricas obtuvo un Accuracy del 97,37 %, una Precision del 100 % y un Recall del 92,86 %, con un F1-score del 96,30 %; además de esto, presentó solamente tres falsos negativos y ningún falso positivo. Aclarando que trabaja con datos relacionados con el diagnóstico de cáncer de mama, es importante enfocarse a la identificación de los casos malignos. Aunque SVM obtuvo exactamente las mismas métricas, Random Forest tiene la capacidad de combinar múltiples árboles de decisión, esto para el fin académico supositorio, no en el ámbito médico real.

## Conclusiones

La comparación permitió evaluar cinco algoritmos de clasificación mediante las métricas Accuracy, Precision, Recall y F1-score. Random Forest y SVM obtuvieron el mejor desempeño general, con una exactitud del 97,37 % y un Recall del 92,86 %. Además, ambos registraron únicamente tres falsos negativos, aspecto importante al identificar tumores malignos.

Se concluye que la selección de un algoritmo depende tanto de sus resultados como del objetivo del problema. En este caso, Random Forest fue seleccionado como modelo final por su buen desempeño general. Sin embargo, los resultados corresponden a un ejercicio académico y no deben utilizarse como herramienta de diagnóstico médico.

## Referencias

Scikit-learn developers. (s. f.). *Breast cancer Wisconsin dataset*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

Scikit-learn developers. (s. f.). *Scikit-learn: Machine learning in Python*. https://scikit-learn.org/stable/

**Enlace al repositorio:** https://github.com/Jhonny12345678-stack/Algoritmos_de_clasificacion__tumores_mama_intro_machine_learning/blob/main/Algoritmos_de_clasificacion.ipynb

---

